EEL 5764: Computer Architecture

Lecture 12: Pipelined Datapath, Hazards & gem5 Simulation Models

Sandip Ray
Warren B. Nelms Endowed Professor
Department of Electrical & Computer Engineering, University of Florida | Office: MALA 4131
Required Reading: Hennessy & Patterson Appendix C, Patterson & Hennessy Chapter 4.5-4.8

From Single-Cycle to Pipelining: Overlapping Execution

THE PARALLELISM REVOLUTION IN CPU DESIGN

SINGLE-CYCLE LIMITATIONS

- **Sequential Execution Bottleneck:** Each instruction must traverse all 5 stages (IF, ID, EX, MEM, WB) sequentially before the next instruction can start.
- **Clock Period Constraint:** Clock frequency is strictly capped by the worst-case critical path (load instructions ~650ps), forcing fast instructions to wait.
- **Idle Hardware Units:** While an instruction is in the MEM stage, the ALU, Register File, and Instruction Memory sit completely idle.

PIPELINED OVERLAPPING

- **Assembly Line Principle:** Multiple instructions execute simultaneously in different pipeline stages, dramatically boosting instruction throughput.
- **Shorter Clock Period:** Clock period shrinks from the sum of all 5 stages (~650ps) down to the latency of the single slowest stage (~200ps).
- **Ideal Speedup:** Under ideal conditions with no stall cycles, a 5-stage pipeline approaches an Ideal CPI of 1.0 with up to 5x throughput speedup.

The 5-Stage RISC-V Pipeline Architecture

1. IF
Instruction Fetch
Read 32-bit instruction from I-Cache using PC; increment PC+4.

2. ID
Instruction Decode
Parse opcode, read rs1/rs2 from Register File, sign-extend immediate.

3. EX
Execute / ALU
Perform ALU operation, calculate effective address or branch target.

4. MEM
Memory Access
Read or write Data Cache for loads (lw) and stores (sw).

5. WB
Write-Back
Write result (ALU or Memory) back to destination register rd.

THE PIPELINE PRINCIPLE
On every active clock edge, all 5 pipeline stages advance simultaneously. Up to 5 distinct instructions occupy the datapath concurrently, separated by stateful Pipeline Registers.

Pipeline Registers: State Separation Between Stages

ISOLATING STAGE BOUNDARIES ON RISING CLOCK EDGES

THE 4 PIPELINE REGISTERS

- **IF/ID Register:** Holds fetched 32-bit instruction word and incremented PC+4 value.
- **ID/EX Register:** Holds read register values (rs1, rs2), sign-extended immediate, PC, and control signals.
- **EX/MEM Register:** Holds ALU calculation result, store data (rs2), zero flag, and target address.
- **MEM/WB Register:** Holds loaded Data Memory value, ALU result, and destination register specifier rd.

STATE PRESERVATION RULES

- **Carrying Control Signals:** Control wires generated in ID must be passed forward through pipeline registers until consumed in EX, MEM, or WB.
- **Destination Register Hazard:** The rd index must travel all the way to MEM/WB so WB knows which register to write back to!
- **Edge-Triggered Sync:** Registers update synchronously on the rising clock edge, preventing data from bleeding into adjacent stages.

Pipelined Performance: Throughput vs. Latency

THROUGHPUT GAINS

- **Definition:** Number of instructions completed per unit time.
- **Pipelining Maximizes Throughput:** By overlapping execution, a new instruction completes every single clock cycle.
- **Ideal CPI = 1.0:** In a pipeline with zero stalls, average CPI reaches 1.0.

LATENCY OVERHEAD

- **Definition:** Time required for a single instruction to complete.
- **Pipeline Register Delay:** Clock period is set by max stage delay plus register setup/hold overhead (t_setup + t_clk-to-q).
- **Unbalanced Stages:** If stages are unbalanced (e.g. MEM = 200ps, ID = 100ps), clock is forced to 200ps.

THE SPEEDUP FORMULA

- **Ideal Speedup = Pipeline Depth (k):** A 5-stage pipeline offers up to 5x speedup.
- **Actual Speedup Formula:** Speedup = (k * CPI_unpipelined) / (1 + Pipeline_Stall_Cycles).
- **Diminishing Returns:** Overhead and hazard stalls prevent reaching theoretical k limit.

Introduction to Pipeline Hazards: The Three Bottlenecks

STRUCTURAL HAZARDS

- **Hardware Resource Conflict:** Occurs when two instructions attempt to use the same physical hardware resource in the same cycle.
- **Classic Memory Conflict:** Trying to fetch an instruction (IF stage) while executing a load/store (MEM stage) on a single unified memory.
- **Solution:** Separate Instruction and Data Caches (Harvard Architecture).

DATA HAZARDS

- **Instruction Data Dependency:** Occurs when an instruction depends on the result of a previous instruction still in the pipeline.
- **Read-After-Write (RAW):** Later instruction tries to read a register before earlier instruction writes it back.
- **Solution:** Hardware Forwarding/Bypassing or Compiler NOP Insertion.

CONTROL HAZARDS

- **Branching Uncertainty:** Occurs when the pipeline must make a control flow decision (beq/bne) before the target PC is calculated.
- **Branch Penalty:** Fetching instructions down the wrong path requires flushing pipeline registers (wasted cycles).
- **Solution:** Dynamic Branch Predictors and Branch Delay Slots.

Data Hazards: Read-After-Write (RAW) Dependencies

WHEN INSTRUCTIONS TRY TO READ DATA BEFORE IT IS WRITTEN

THE RAW DEPENDENCY SCENARIO

- **Example Code Sequence:**
- sub x2, x1, x3 # Calculates x2 in EX, writes back in CC5
- and x12, x2, x5 # Tries to read x2 in ID during CC3!
- or x13, x6, x2 # Tries to read x2 in ID during CC4!
- **The Register File Timing Gap:** sub produces its result at the end of CC3 (EX stage), but doesn't write to x2 until CC5 (WB stage).
- **Data Corruption Threat:** Without hardware intervention, and and or will read the stale, old value of x2 from the Register File.

SOFTWARE NO-OP SOLUTION

- **Compiler NOP Insertion:** The compiler can insert 2 explicit nop instructions (addi x0, x0, 0) between dependent instructions.
- **Execution Delay Cost:** NOPs stall the pipeline for 2 full clock cycles, raising CPI and wasting execution time.
- **Why Hardware Solutions Win:** Inserting software NOPs bloats binary code size and wastes execution energy, motivating hardware forwarding.

Hazard Mitigation: Forwarding, Stalls, & Branch Prediction

HARDWARE FORWARDING

- **Bypassing Register File:** Route the calculated result directly from EX/MEM or MEM/WB pipeline registers back to ALU inputs.
- **Zero-Stall ALU Operations:** Completely eliminates RAW stalls for consecutive register-register arithmetic instructions.
- **Forwarding Unit:** Hardware comparator checking if (EX/MEM.rd == ID/EX.rs1 or rs2).

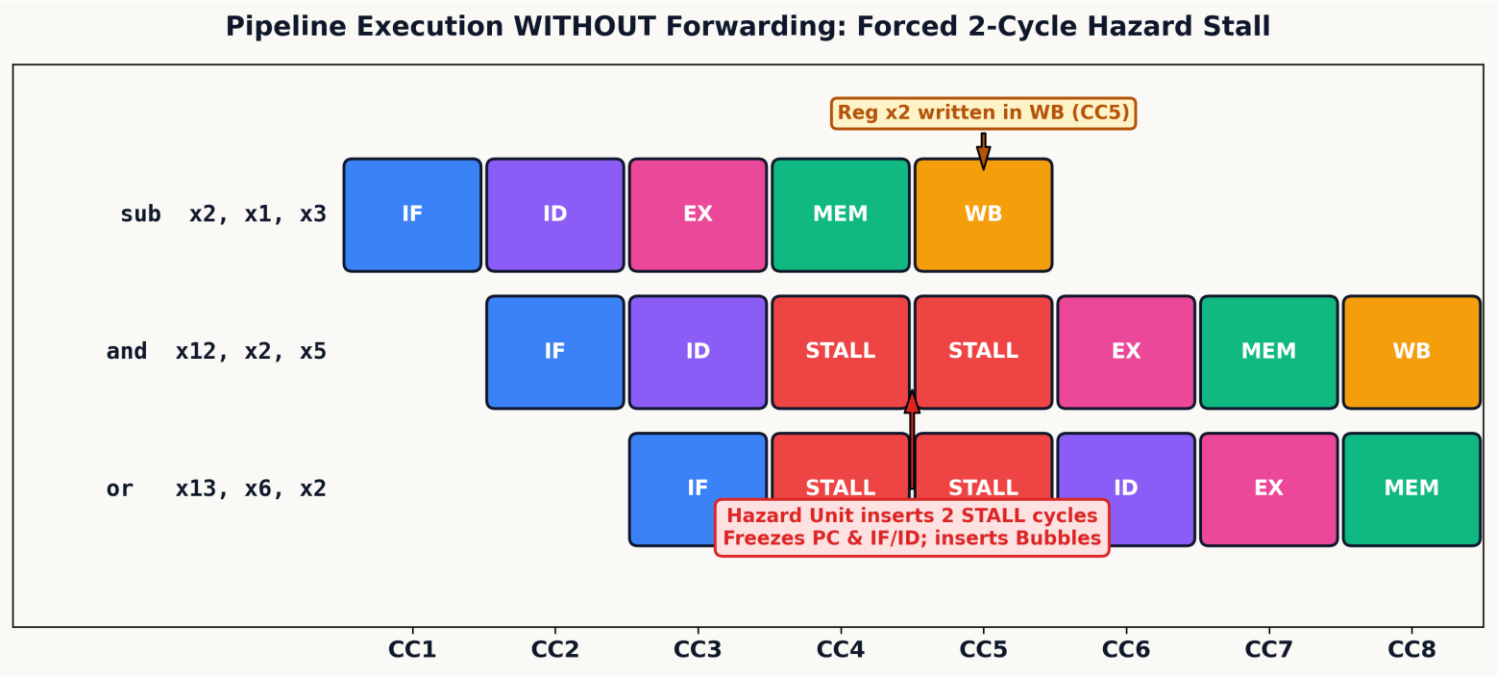
LOAD-USE STALLS

- **The Load Delay Slot:** Memory load data (lw) is not available until the end of the MEM stage (CC4).
- **Unavoidable 1-Cycle Bubble:** An arithmetic instruction immediately following a load that uses the load result MUST stall 1 cycle.
- **Hardware Hazard Detection Unit:** Detects (ID/EX.MemRead and (ID/EX.rd == IF/ID.rs1 or rs2)).

BRANCH PREDICTION

- **Predicting Control Flow:** Guess whether conditional branches are Taken or Not-Taken during the IF stage.
- **Static vs. Dynamic:** Static predicts backward branches Taken (loops), forward Not-Taken; Dynamic uses 2-bit branch history tables.
- **Pipeline Flush:** If prediction is wrong, zero out control signals in IF/ID and ID/EX (1 to 2-cycle penalty).

Pipelined Datapath Without Data Forwarding: The Stall Penalty



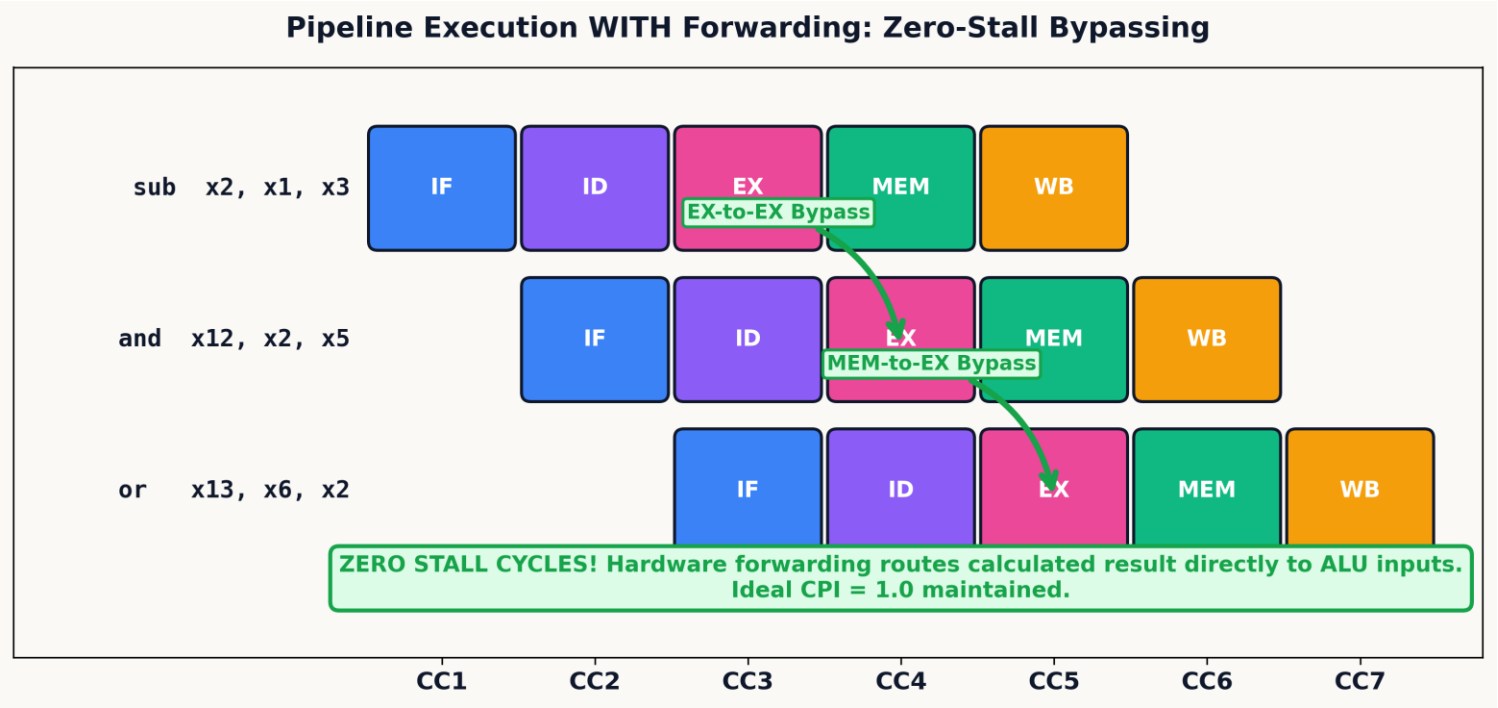
The Timing Gap: sub calculates x2 in CC3 (EX), but does not write back to x2 until CC5 (WB).

Hardware Intervention: Hazard Detection Unit freezes PC and IF/ID registers for 2 full clock cycles (CC3 and CC4).

Bubble Insertion: Hardware injects zeroed control signals (NOP bubbles) into EX/MEM stage.

Performance Penalty: Adds +0.67 stall cycles per dependent instruction block, degrading CPI.

Pipelined Datapath With Data Forwarding: The Bypass Pathway



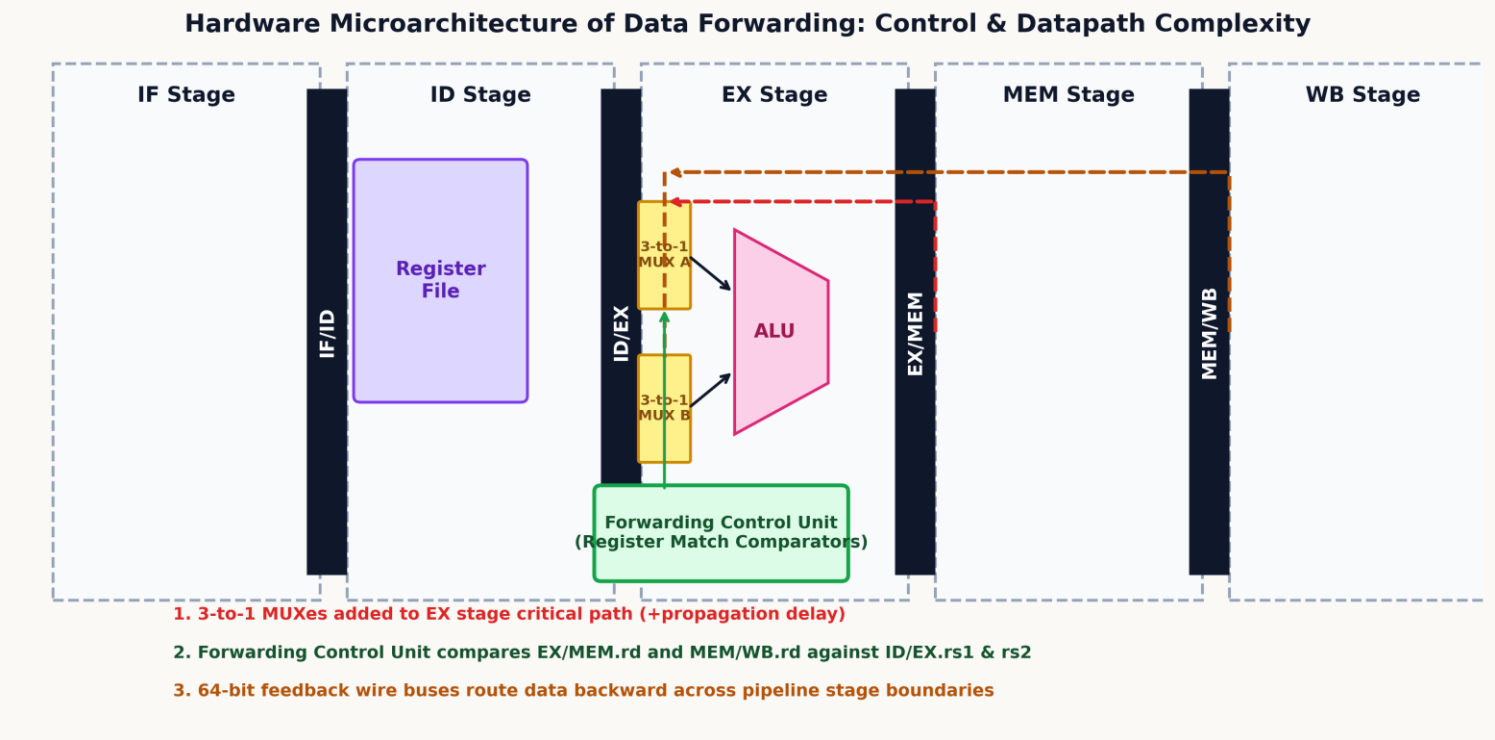
EX-to-EX Bypass (CC4): Result in EX/MEM register is routed directly to ALU input for and in CC4.

MEM-to-EX Bypass (CC5): Result in MEM/WB register is routed directly to ALU input for or in CC5.

Zero-Stall Execution: Completely eliminates RAW stalls for consecutive arithmetic instructions.

Maintained CPI: Preserves Ideal CPI = 1.0 execution throughput.

Hardware Complexity Overhead of Data Forwarding



- **3-to-1 Multiplexers:** Replaces 2-to-1 MUXes at both ALU inputs, adding MUX gate delay to critical EX path.
- **Forwarding Control Unit:** 5-bit register comparators inspecting EX/MEM.rd and MEM/WB.rd against ID/EX.rs1/rs2.
- **Parallel Feedback Buses:** 64-bit bypass wire traces routing backward across physical stage boundaries.
- **Area & Delay Penalty:** Increases silicon area, wire congestion, and EX stage setup time.

Control Hazards: Branch Penalty & Pipeline Flushes

MANAGING CONTROL FLOW UNCERTAINTY IN PIPELINES

THE BRANCH PENALTY PROBLEM

- **Branch Decision Timing:** In a basic pipeline, the branch condition (beq) and target address are calculated in the EX stage (CC3).
- **Speculative Fetching:** While the branch is evaluated, the CPU fetches the next sequential instructions (PC+4, PC+8).
- **The Penalty on Taken Branches:** If the branch is TAKEN, the 2 speculatively fetched instructions in IF and ID are invalid and must be discarded.

PIPELINE FLUSH MECHANICS

- **Flushing (Bubble Insertion):** Convert invalid instructions in IF/ID and ID/EX registers into NOPs by clearing control bits to zero.
- **Early Branch Resolution:** Moving branch comparison logic into the ID stage reduces taken branch penalty from 2 cycles down to 1 cycle.
- **Impact on CPI:** If branches represent 20% of code and 50% are taken with a 2-cycle penalty, CPI increases by 0.20 * 0.50 * 2 = +0.20.

gem5 CPU Models: From Functional Abstraction to Detailed Pipeline

THE GEM5 SIMULATION MODEL SPECTRUM

gem5 provides a progression of CPU models trading off simulation speed for microarchitectural cycle accuracy. Choosing the correct CPU model is critical for graduate architecture research.

AtomicSimpleCPU

- **Functional Execution:** Executes every instruction in a single atomic memory transaction (1 cycle).
- **No Pipeline or Cache Modeling:** Unrealistic CPI=1.0. Used for fast booting OS kernels or functional code checks.

TimingSimpleCPU

- **Detailed In-Order Modeling:** Simulates detailed timing for cache misses, memory stalls, and resource contention.
- **Stalls on Memory:** Models in-order execution with realistic stall cycles. Used in gem5 lab exercises.

O3CPU (Out-of-Order)

- **Full Cycle-Accurate Pipeline:** Simulates 7-stage out-of-order execution, reorder buffers (ROB), issue queues, and branch predictors.
- **Detailed Telemetry:** Captures exact pipeline stalls, hazard stalls, and dynamic execution behavior.

gem5 Pipelined Simulation Launch & Command-Line Flags

TIMING SIMPLE IN-ORDER LAUNCH

```
   # Launch TimingSimpleCPU with L1 Caches enabled
build/RISCV/gem5.opt configs/deprecated/example/se.py
\
    --cpu-type=TimingSimpleCPU \
    --caches \
    --l1d_size=64kB --l1i_size=32kB \
    --cmd=tests/test-
progs/hello/bin/riscv/linux/hello
```

- **--cpu-type=TimingSimpleCPU:** Invokes detailed in-order memory stall modeling.
- **--caches:** Instantiates L1 Data and Instruction Caches.
- **SE Mode:** Executes benchmark binary in Syscall Emulation mode.

DETAILED O3CPU PIPELINE LAUNCH

- **--cpu-type=O3CPU:** Activates full 7-stage out-of-order execution model.
- **--bp-type=TournamentBP:** Enables advanced dynamic branch predictor.
- **m5out/ Directory:** All output statistics write automatically to stats.txt.

```
   # Launch O3CPU with Tournament Branch Predictor
build/RISCV/gem5.opt
configs/deprecated/example/se.py \
    --cpu-type=O3CPU \
    --bp-type=TournamentBP \
    --caches \
    --cmd=tests/test-
progs/hello/bin/riscv/linux/hello
```

Parsing gem5 Pipeline Stats: Telemetry in m5out/stats.txt

Key cycle-level performance counters extracted from gem5 simulation outputs:

gem5 Stat Metric String	Architectural Description	Formula / Significance
`sim_ticks`	Total simulated execution runtime in picoseconds	Divide by 10^12 for simulated seconds
`system.cpu.numCycles`	Total processor clock cycles spent executing workload	Total Clock Cycles (CC)
`system.cpu.cpi / ipc`	Cycles Per Instruction or Instructions Per Cycle	CPI = numCycles / committedInsts
`system.cpu.dcache.overall_misses`	Raw count of L1 Data Cache misses	Triggers high-latency memory stalls
`system.cpu.dcache.overall_miss_rate`	L1 Data Cache miss rate (misses / total accesses)	Used to compute Effective CPI overhead
`system.cpu.branchPred.lookups`	Total conditional branch instructions evaluated	Total branch count
`system.cpu.branchPred.condIncorrect`	Count of mispredicted branches (flushes triggered)	Misprediction Rate = condIncorrect / lookups

gem5 Case Study: In-Order vs. Out-of-Order Execution Delta

TIMING SIMPLE IN-ORDER

- **Execution Model:** In-Order TimingSimpleCPU.
- **CPI Result:** CPI = 1.68.
- **Primary Bottleneck:** Memory stalls from cache misses block all subsequent instructions in the pipeline, raising CPI significantly above 1.0.

O3CPU OUT-OF-ORDER

- **Execution Model:** Out-of-Order O3CPU.
- **CPI Result:** CPI = 0.82 (IPC = 1.22).
- **Performance Gain:** Dynamic instruction scheduling executes independent instructions around memory stalls, achieving superscalar IPC > 1.0!

THE QUANTITATIVE DELTA

- **Speedup Delta:** O3CPU delivers 2.05x speedup over TimingSimpleCPU.
- **Architectural Takeaway:** Pipelining alone (in-order) is bounded by memory stalls; out-of-order execution hides latency by exploiting ILP.

Calculating Pipelined CPI with Stalls

QUANTITATIVE EVALUATION OF PIPELINE HAZARD STALLS

THE WORKLOAD & PIPELINE PROFILE

- **Baseline Pipeline:** 5-stage pipeline operating at clock frequency f = 2.0 GHz.
- **Instruction Mix:**
- • 60% Integer Arithmetic / Logic (0 stall cycles)
- • 20% Load Instructions (25% followed immediately by load-use dependent instruction requiring 1 stall)
- • 20% Branch Instructions (60% taken, triggering a 2-cycle flush penalty)
- **Challenge Goal:** Calculate the Effective CPI of the pipeline and the total execution time for a program with IC = 10^9 instructions.

STEP-BY-STEP SOLUTION

- **1. Calculate Stall Contributions per Instruction Class:**
- • Arithmetic Stall = 0.60 * 0 = 0.00 cycles
- • Load Stall = 0.20 * (0.25 * 1 stall) = 0.05 cycles
- • Branch Stall = 0.20 * (0.60 * 2 stalls) = 0.24 cycles
- **2. Sum Total Stall Cycles per Instruction:**
- • Total Stall Overhead = 0.00 + 0.05 + 0.24 = 0.29 cycles
- **3. Calculate Effective CPI:**
- • Effective CPI = Ideal CPI (1.0) + Total Stalls = 1.0 + 0.29 = 1.29 CPI
- 4. Execution Time = IC * CPI / f = (10^9 * 1.29) / (2.0 * 10^9 Hz) = 0.645 seconds!